

Lesson 5 — Tilemaps

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The level

```
const TILE = 32;
const LEVEL = [
  "#####",
  "#.....#",
  "#.###...###....#",
  "#####",
];
```

The two conversions

```
col = Math.floor(x / TILE); // position → square
x   = col * TILE;          // square → position
```

floor, not round. `floor(-1.2) = -2`, which is right; chopping gives `-1`, which lets the player stand in the left wall.

Reading it safely

```
function tileAt(col, row) {
  if (col < 0 || row < 0 ||
      row >= LEVEL.length ||
      col >= LEVEL[0].length) return "#";
  return LEVEL[row].charAt(col);
}
```

Out of bounds counts as solid. One decision, a whole class of bug gone.

Tiles as a table, not as `if s`

```
const TILES = {
  ".": { solid: false, colour: "#121820" },
  "#": { solid: true,  colour: "#3c4756" },
  "^": { solid: false, deadly: true  }
};
```

A new tile type is one line. The drawing code never grows.

Which tiles does the box touch?

```
c0 = floor( box.x           / TILE);
c1 = floor((box.x + box.w - 0.001) / TILE);
r0 = floor( box.y           / TILE);
r1 = floor((box.y + box.h - 0.001) / TILE);
```

At most **4 tiles**, whatever the size of the level. The `- 0.001` stops a box whose edge is exactly on a boundary from counting the next column — without it the player sticks in gaps exactly their own width.

Collision: TWO steps, never one

```
// X
x += vx * dt;
if (hit) {
  x = vx > 0 ? hit.col * TILE - w
    : (hit.col + 1) * TILE;

  vx = 0;
}
// Y
y += vy * dt;
if (hit) {
  if (vy > 0) { y = hit.row * TILE - h;
    onGround = true; }
  else      { y = (hit.row + 1) * TILE; }
  vy = 0;
}
```

One step cannot know **which** direction to undo. Two steps always know.

Sliding along walls comes free. `onGround` comes out of the **downward** test.

Only draw what is visible

```
firstCol = floor(camera.x / TILE);
lastCol  = floor((camera.x + W) / TILE) + 1;
```

Section A — Recall

A1. Give three advantages a grid has over a list of arbitrary rectangles. [3 marks]

A2. Why `Math.floor` rather than `Math.round`, and why rather than chopping off the decimals? [2 marks]

A3. Why does `tileAt` return a wall for positions off the map? Name two problems that removes. [2 marks]

A4. Why must collision be resolved one axis at a time? What information does the one-step version not have? [3 marks]

A5. Where does `onGround` come from, and why is it not something you set when a key is pressed? [2 marks]

A6. What is the `- 0.001` for? What is the symptom without it? [2 marks]

A7. Why put tile properties in a table instead of using `if` statements? [2 marks]

Section B — Predict the output

B1. `TILE` is 32. Which column and row is the point (95, 64) in? What is the top-left corner of that square? [3 marks]

B2. A player box is at $x = 60$, $y = 30$, 26 wide and 26 tall. `TILE` is 32. Which columns and rows does it touch? How many squares is that? [4 marks]

B3. Same player, now at exactly $x = 64$. With the `- 0.001`, which columns does it touch? Without it? [3 marks]

B4. A player 26 tall is falling with `vy = 500` and `dt = 0.1`, starting at $y = 100$. There is a floor of solid tiles in row 5. Where do they end up, and what is `vy` afterwards? (`TILE = 32`.) [4 marks]

B5. A level is 300×300 tiles and the screen shows 25×16 of them. How many tiles does the naive draw loop visit per frame? How many does the culled one? [3 marks]

Section C — Find and fix the bug

C1. The player sinks exactly 16 pixels into the floor. `TILE` is 32. [3 marks]

```
const row = Math.round(y / TILE);
```

C2. Walking diagonally into a corner makes the player climb the wall. Name the bug and describe the fix. [4 marks]

```
player.x += player.vx * dt;
player.y += player.vy * dt;
const hit = findSolidOverlap(player);
if (hit) {
  player.x -= player.vx * dt;
  player.y -= player.vy * dt;
}
```

C3. The console says `Cannot read properties of undefined (reading 'charAt')` when the player reaches the top of the level. [3 marks]

```
function tileAt(col, row) {
  return LEVEL[row].charAt(col);
}
```

C4. The player can jump for ever by holding into a wall. Both halves of this snippet contribute — name them. [4 marks]

```
const hit = findSolidOverlap(player);
if (hit) {
  player.onGround = true;
  player.vy = 0;
}
```

C5. The right-hand quarter of the level has no walls in it at all, although the author typed them. The level loads with no error. [3 marks]

```
const LEVEL = [
  "#####",
  "#.....#",
  "#.###.###.",
  "#####",
];
```

Section D — Build it

- ☐ **Checkpoint 1.** Write your level as an array of strings and draw it with coloured rectangles. Add a check at startup that every row is the same length, and make it complain loudly if not.
- ☐ **Checkpoint 2.** Write `tileAt` with the out-of-bounds rule, and `isSolid` using a `TILES` table. Print the column and row under the mouse pointer so you can see the conversion working.
- ☐ **Checkpoint 3.** Add a player box that moves with the arrow keys. No collision yet — walk through walls on purpose, and draw an outline on every tile your box is touching. You should see at most four.
- ☐ **Checkpoint 4.** Add the two-step collision. Test: walk into a wall head-on, then walk into it diagonally. You should slide along it without writing any sliding code.
- ☐ **Checkpoint 5.** Add gravity and make `onGround` come out of the downward test. Draw `onGround` on screen as a word. Then add a jump that only works when it is true.
- ☐ **Checkpoint 6.** Add two more tile types from your table — spikes that reset the player and coins that score. Neither should need a change to the drawing code.
- ☐ **Checkpoint 7.** Put the level in a `<textarea>` and reload it as you type. Then redesign your level by typing.
- ☐ **Checkpoint 8.** Add the broken one-step collision on a key, so you can demonstrate the difference to somebody.

IF THE PLAYER STICKS IN GAPS

Print `c0`, `c1`, `r0` and `r1` on screen and walk

slowly. If `c1` jumps forward one square before the player has actually crossed into it, you need the `- 0.001`.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. The tile table holds *properties*. Where does that approach break down? Think about a tile that needs *behaviour* — a door that opens, a platform that moves, a button that does something when stood on.

E2. Our level is an array of strings. Alternatives: a flat array of numbers, an array of arrays, one long string with newlines. What does each make easy, and awkward? Which would you pick for a level 10,000 tiles across, and why?

E3. Design a one-way platform: solid from above, pass-through from below. What must the collision code know that it currently does not?

E4. The hard one. 200 coins, and collecting one changes the map — but the map is also what you draw and what you collide against. Where does "this coin is taken" live? Change the level array, keep a separate list, or keep two layers? Pick one, name its cost, and say what happens when the level restarts.

Stretch goals

1. One-way platforms (E3). One condition.
2. Draw tiles from a tileset built in code, as in lesson 4.
3. Auto-tiling: pick a wall's picture from which of its four neighbours are walls.
Sixteen combinations — find the pattern rather than writing sixteen cases.
4. A level editor: click to place, right-click to erase, and a button that prints the level back out as text.
5. Slopes. Genuinely hard. Attempt it and write down exactly which assumption breaks.